```
import streamlit as st
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd

def create_network(users, connections):
    G = nx.Graph()
    for user in users:
        G.add_node(user)
    for connection in connections:
        G.add_edge(connection[0], connection[1])
    return G

def calculate_centrality(G):
    centrality = nx.degree_centrality(G)
    return centrality

def draw_graph(G, title):
    plt.figure()
    pos = nx.spring_layout(G)
    nx.draw(
        G,
        pos,
        with_labels=True,
        node_color='skyblue',
        node_size=2000,
        font_size=10,
        font_weight='bold'
```

```
    )
    plt.title(title)
    st.pyplot(plt)

def plot_bar_chart(centrality, title):
    plt.figure()
    users = list(centrality.keys())
    values = list(centrality.values())
    plt.bar(users, values)
    plt.xlabel("Users")
    plt.ylabel("Centrality Score")
    plt.title(title)
    st.pyplot(plt)

st.set_page_config(page_title="Influencer Detection", layout="centered")

st.title("📊 Social Media Influencer Detection")
st.write("MMCA Project using Degree Centrality Model")

st.sidebar.header("⚙️ Input Parameters")

num_users = st.sidebar.slider("Number of Users", 5, 10, 7)

users = [chr(65 + i) for i in range(num_users)]

st.sidebar.write("Users:", users)

connections = [
```

```
    ('A', 'B'),
    ('A', 'C'),
    ('B', 'C'),
    ('C', 'D'),
    ('D', 'E'),
    ('E', 'F'),
    ('F', 'G'),
    ('C', 'G')
]

G = create_network(users, connections)

centrality = calculate_centrality(G)

st.subheader("📌 Centrality Values")

for user, value in centrality.items():
    st.write(f"User {user} → {value:.2f}")

influencer = max(centrality, key=centrality.get)

st.success(f"🔥 Main Influencer: User {influencer}")

st.subheader("📈 Network Graph")

draw_graph(G, "Social Network Structure")

st.subheader("📊 Centrality Comparison")
```

```
plot_bar_chart(centrality, "Influencer Detection using Degree Centrality")

st.subheader("📋 Centrality Table")

df = pd.DataFrame({
    "User": list(centrality.keys()),
    "Centrality Score": list(centrality.values())
})

st.dataframe(df)

st.subheader("🔁 Scenario 2: Adding More Connections")

if st.button("Add New Connections (A-D, A-E)"):
    G.add_edge('A', 'D')
    G.add_edge('A', 'E')
    new_centrality = calculate_centrality(G)
    st.write("### Updated Centrality")
    for user, value in new_centrality.items():
        st.write(f"User {user} → {value:.2f}")
    new_influencer = max(new_centrality, key=new_centrality.get)
    st.success(f"🔥 New Influencer: User {new_influencer}")
    draw_graph(G, "Updated Network Structure")
    plot_bar_chart(new_centrality, "Updated Centrality After Changes")
    df2 = pd.DataFrame({
        "User": list(new_centrality.keys()),
        "Centrality Score": list(new_centrality.values())
```

```
    })
    st.dataframe(df2)

st.subheader("📘 Mathematical Model")

st.latex(r"C(v) = \frac{deg(v)}{N-1}")

st.markdown("""
Where:
• C(v) = Centrality of user
• deg(v) = Number of connections
• N = Total users
Higher value means higher influence.
""")

st.subheader("💡 Interpretation")

st.write("""
• Users with more connections have higher influence
• Central node becomes main influencer
• Adding connections increases influence
• Network structure affects results
""")

st.subheader("📌 Conclusion")

st.write("""
This model successfully identifies influencers in a social network using degree centrality.
```

Users with maximum connections act as key influencers.

The system dynamically updates when connections change.

""")